

Q2 print numbers from 1 to n in increasing order.

way 1

```
f(i, n) { // i = 1
    if (i > n) {
        return;
    }
    print(i);
    f(i+1, n);
}
```

case (2) Approach
post-operation

Q3 factorial of number :

```
fact(n) {
    if (n == 0) {
        return 1;
    }
    return n * fact(n-1);
}
```

Q4 Sum of first n natural numbers.

```
func(n) {
    if (n == 0) {
        return 0;
    }
    return n + func(n-1);
}
```

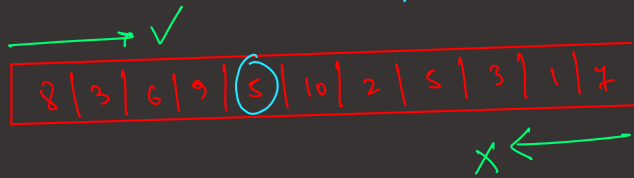
Q5 find nth fibonacci number.

```
fib(n) {
    if (n == 0 || n == 1) { return n; }
    return fib(n-1) + fib(n-2);
}
```

C.C =

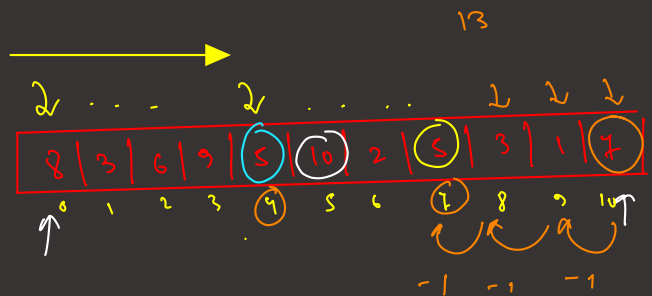
E.C =

hw Write a fn to find first occurrence of an element in an array $\hookrightarrow 5$



Q Similarly, the last occurrence

Q Find n^{th} (i) $O(n)$
(ii) $O(\log n)$



day-16

Last Occurrence

```
int lastOccurrence (int arr[], int i, int key) {
```

```
    if (i == arr.length) {
```

```
        return -1; ✓
```

```
    }
```

```
    int isFound = lastOccurrence (arr, i+1, key)
```

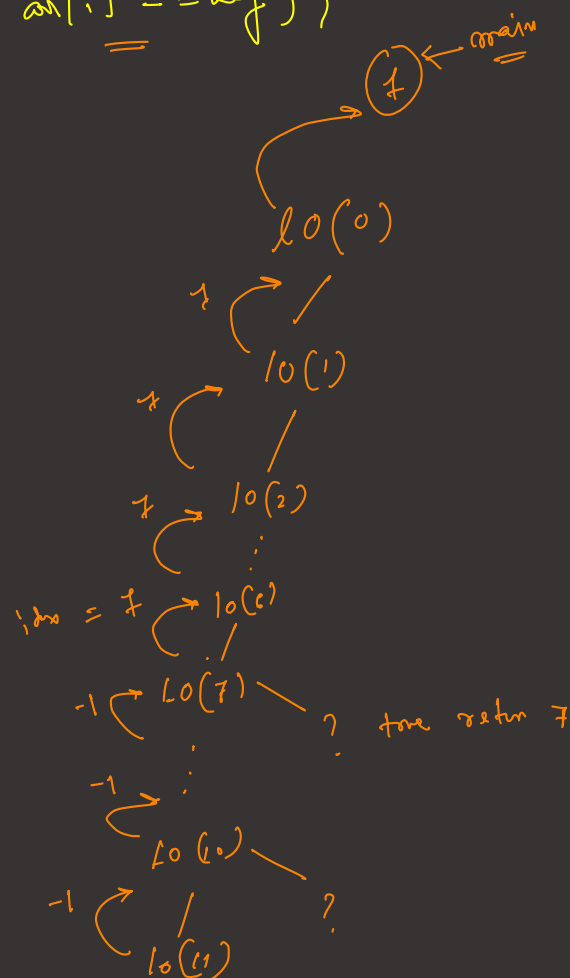
```
    if (isFound == -1 && arr[i] == key) {
```

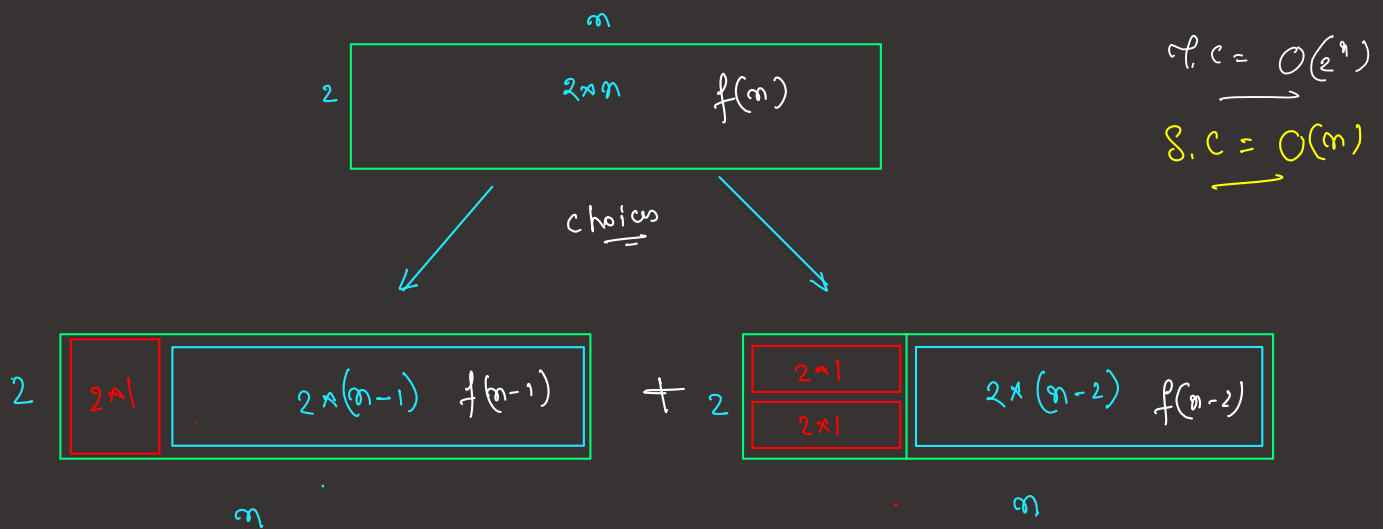
```
        return i;
```

```
    }
```

```
    return isFound; ✓
```

```
}
```





Problem : Friends Pairing Problem

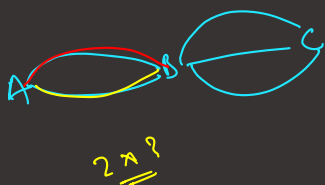
Given n friends, each one can remain single or can be paired up with some other friend. Each friend can be paired up only once. Find total no. of ways in which friends can remain single or can be paired up.

$n=1$ (single) ways = 1

$n=2$ a, b ways = 2
 (a, b)

$n=3$

a (single), $\{b, c\}$
 $\left\{ \begin{array}{l} a, b, c \\ a, (b, c) \end{array} \right\} \quad 2$



a (paired)
 $\left\{ \begin{array}{l} b \\ c \end{array} \right\} \quad 2$
 $(a, b) \quad c \quad (a, c) \quad b$

total = 4 ways

$$f(n) = f(n-1) + (n-1) \cdot f(n-2)$$

$\text{r.c} = O(2^n)$

Given $\delta m =$ ~~suvaait sadhulalshon~~

Ans = Savajitdhhn

$$\begin{aligned} 'a' - 'a' &= 0 \\ 'b' - 'a' &= 1 \\ 'c' - 'a' &= 2 \end{aligned}$$

```

8thrg Duplicates( str, i, newStr, map[1]) {
    if ( i == str.length ) {
        return newStr;
    }
    if ( map[ str.charAt(i) - 'a' ] )
        Duplicates( str, i+1, newStr, map );
}

```

use String Builder

```

    map[ str.charAt(i) - a ] = row;
    newStr = newStr + str.charAt(i);
    Duplicate ( str, i+1, newStr, map );
}

```

^a Surajit Sarker ¹

$$f(0)$$

Q Binary String Problem

Print all binary string of size N without consecutive one.

$N=1$ o/p = 0

$N=2$ o/p = $\left. \begin{array}{l} 00 \\ 01 \\ 10 \\ 11X \end{array} \right\}$

$N=3$, o/p : $\left. \begin{array}{l} 000 \\ 001 \\ 010 \\ 100 \\ 101 \\ 110X \end{array} \right\} S.$

110X

①0

binaryStringSequence(int n, char last, string bstr)

if (n == 0)

s.out(bstr);

return;

}

if (last == 0)

binaryStringSequence(n-1, '0', bstr + "0");

binaryStringSequence(n-1, '1', bstr + "1");

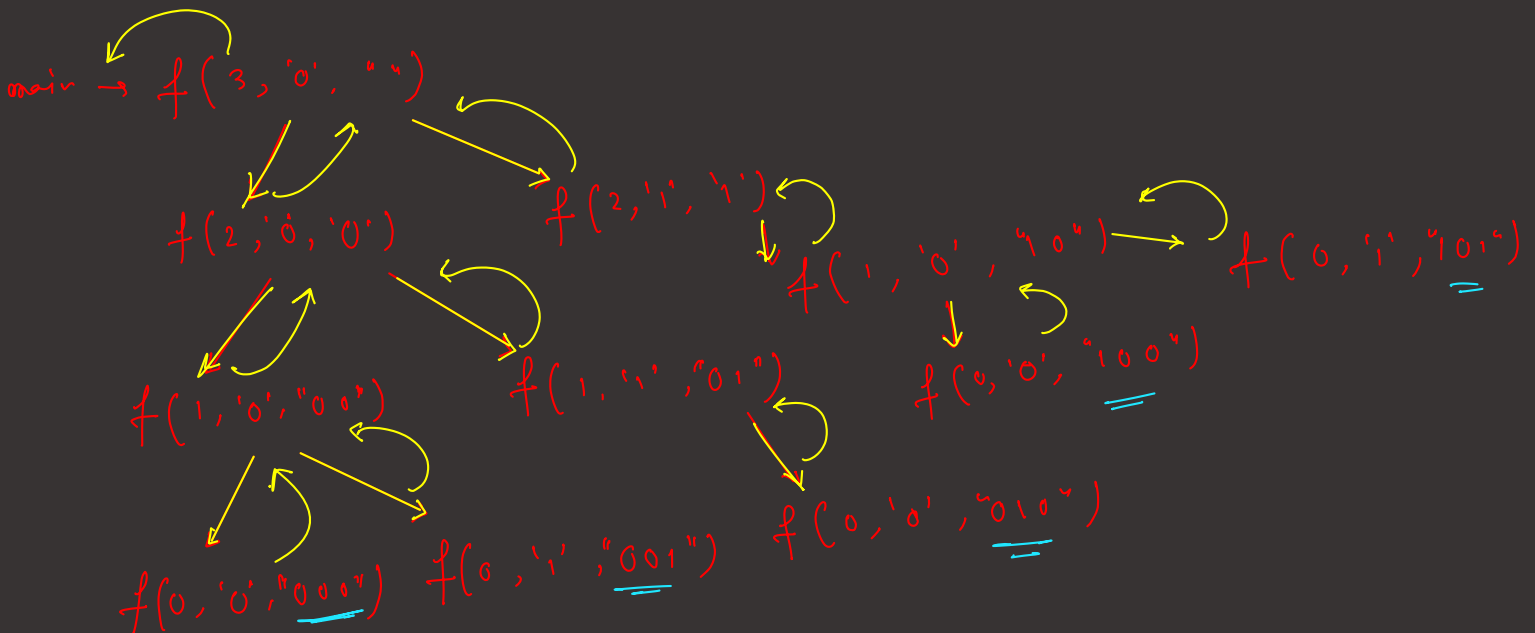
}

else // last == 1

binaryStringSequence(n-1, '0', bstr + "0");

}

}



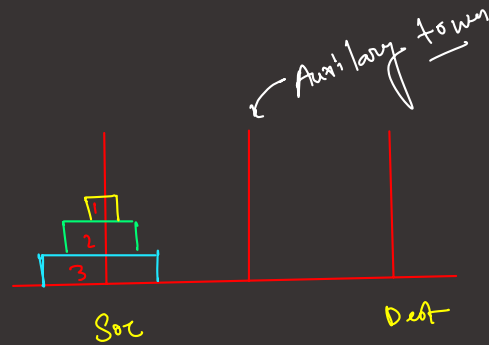
Q Tower of Hanoi

"The Tower of Hanoi, is a mathematical problem which consists of three rods and multiple disks. Initially, all the disks are placed on one rod, one over the other in ascending order of size similar to a cone-shaped tower."

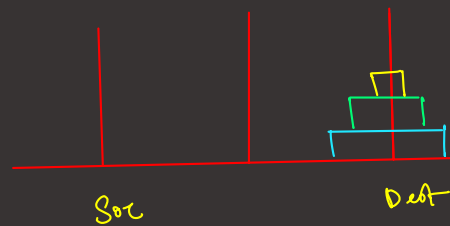
The objective of this problem is to move the stack of disks from the initial rod to another rod, following these rules:

1. Move one disk at a time.
2. No disk can be placed on top of smaller disk.
3. Only the top disk may be moved.

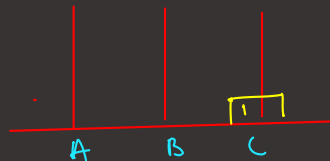
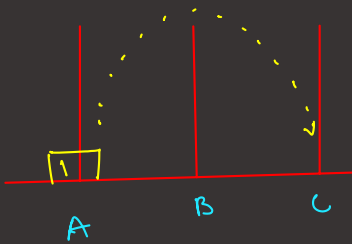
initial :



final :



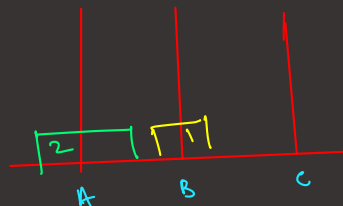
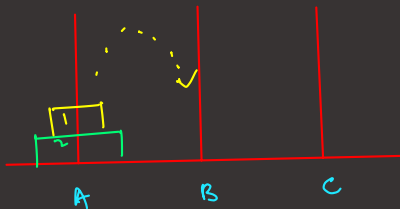
N=1



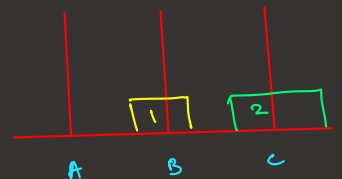
disk 1 moves from 'A' to 'C'

①

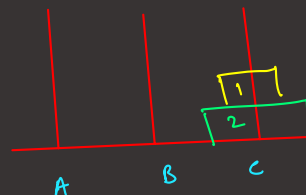
N=2



②

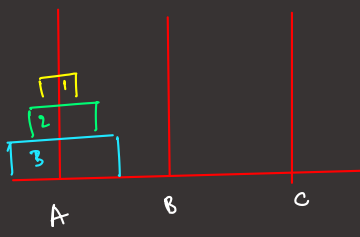


③

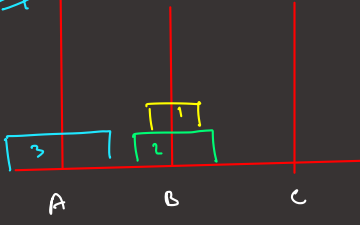


N=3

$f(3)$



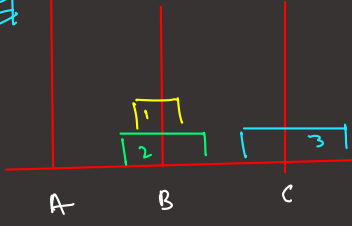
Step 1



'1, 2' A $\rightarrow$ B using 'C'

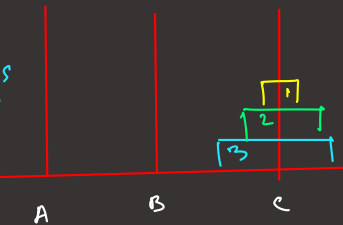
3 steps

Step 2



disk '3' moves from A to C

Step 3



3 step

disk (2, 1) $\rightarrow$ from B to C using 'A'

src dest Auxiliary tower
 $f(n, A, C, B)$

① $f(n-1, A, B, C)$ $\rightarrow$ k steps

② disk n moves from 'src' to 'dest' = 1 step

③ $f(n-1, B, C, A)$ $\rightarrow$ k steps

Disk 1 moves from A to C

Disk 2 moves from A to B

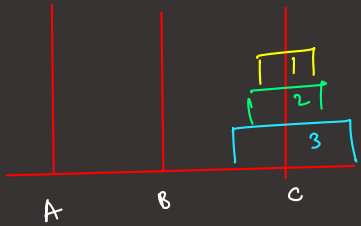
Disk 1 moves from C to B

Disk 3 moves from A to C ✓

Disk 1 moves from B to A

Disk 2 moves from B to C ✓

Disk 1 moves from A to C



Analysis

① Recursion Tree

② Stack analysis

③ a.p.e. and s.p.e